



*Projet : Développement d'un système de navigation autonome
pour les drones Tello*



Sommaire

1. Introduction.....	3
1.1. Présentation du projet.....	3
1.2. Objectifs généraux et contexte.....	3
2. Prise en main du drone Tello et SDK.....	5
2.1. Description des drones Tello.....	5
2.2. Utilisation du SDK Tello pour le contrôle via Python.....	5
3. Conception et impression 3D des cadres de protection.....	7
3.1. Outils et logiciels utilisés pour la conception.....	7
3.2. Processus d'impression 3D : choix des matériaux et validation des modèles.....	8
4. Développement de systèmes de navigation autonome.....	11
4.1. Suivi de trajectoire.....	11
4.2. Reconnaissance d'objets : principes et algorithmes utilisés.....	14
5. Implémentation et tests en conditions réelles.....	18
5.1. Mise en place des scénarios de vol d'essai.....	18
5.2. Analyse des performances et résultats obtenus.....	19
5.3. Réglages et optimisations pour les améliorations futures.....	20
6. Conclusion.....	22
6.1. Résumé des résultats.....	22
6.2. Perspectives d'évolution du projet.....	22
7. Ressources.....	24

Introduction



Dans un contexte où les technologies de drones connaissent une croissance rapide, ce projet se concentre sur l'utilisation des drones Tello pour développer et tester un système de navigation autonome.

Les drones Tello, fabriqués par DJI, sont des appareils légers, programmables et adaptés à des missions d'exploration en intérieur et en extérieur.

Leur simplicité d'utilisation et leur compatibilité avec différents environnements de programmation en font des outils idéaux pour la recherche et le développement de systèmes autonomes.

Les objectifs spécifiques du projet incluent :

- ❖ Prise en main des drones Tello et de leur SDK : Cela comprend l'apprentissage et l'utilisation du kit de développement logiciel (SDK) des drones Tello pour le contrôle via le langage de programmation Python, permettant une manipulation fine et automatisée des drones.
- ❖ Conception et impression 3D des cadres de protection : Afin de sécuriser les vols d'essai, des cadres de protection pour les hélices seront conçus et imprimés en 3D.

Ces structures visent à minimiser les risques d'endommagement des drones lors des tests, notamment en environnement clos.

- ❖ Développement et implémentation des systèmes de navigation autonome : En utilisant des algorithmes de suivi de trajectoire ou de reconnaissance d'objets, des systèmes de navigation autonomes seront développés pour permettre aux drones de fonctionner de manière autonome, en détectant et en évitant les obstacles.
- ❖ Test et validation des performances en conditions réelles : Les drones seront mis à l'épreuve dans des environnements réels afin de valider les performances des systèmes développés.

Ces tests permettront d'identifier les améliorations possibles et d'ajuster les systèmes pour des performances optimales.

Le matériel utilisé dans ce projet :

Matériel	Documentation	Application
Drones Tello DJI	Caméra HD 720p (avec un capteur de 5 méga pixel) Batterie LiPo	
Ordinateur	Ordinateur Personnel ! Visual Studio Code	Langages : Python (DJITelloPy et OpenCV) Outils de conception CAO : FreeCad et Blender IDE : Visual Studio Code Partage de fichiers : GIT
Imprimante 3D	Imprimante 3D de l'école !	EpilogLaser : Création des cadres de protection avec des & filaments 3D

Tableau 1 : Matériel utilisé pour ce projet

Ce projet vise ainsi à explorer et à repousser les limites de l'automatisation des drones dans des environnements variés, tout en garantissant une sécurité et une efficacité maximales lors des vols d'essai.

Prise en main des drones Tello et SDK

1. Description des drones Tello

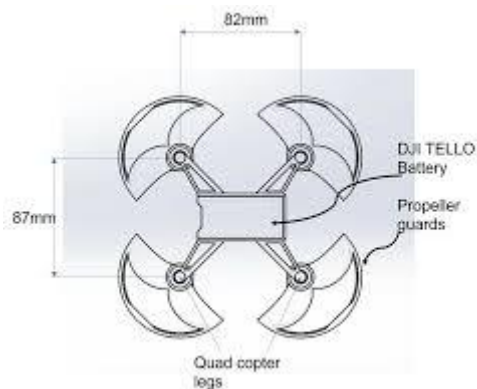


Figure 1 : Ressource dimensions

Les drones Tello, développés par Ryze Tech en partenariat avec DJI, sont des drones compacts, légers et abordables, conçus principalement pour des applications éducatives et expérimentales.

Spécificités :

Poids	87 g
Batterie Tension & Ampérage	3.8V & 1100mAh
Batterie	LiPo
Poids Batterie	25±2 g
Hélices	4
Température	0 à 40°C
Vitesse Max	28.8 km/h
Autonomie (temps de vol)	13 minutes
Caméra taille	2592x1936

Malgré leur petite taille, les drones Tello offrent une grande polyvalence grâce à leur compatibilité avec un kit de développement logiciel (SDK), qui permet aux utilisateurs de les programmer via des langages comme Python.

Ce drone est également doté de fonctionnalités telles que la capture de photos, l'enregistrement vidéo, ainsi que des modes de vol préprogrammés (comme le décollage et l'atterrissage automatique). Son faible coût et sa robustesse en font un outil idéal pour les développeurs débutants, les étudiants et les chercheurs souhaitant explorer la programmation de drones autonomes.

2. Utilisation du SDK Tello pour le contrôle via Python

Le SDK Tello (Software Development Kit) permet de prendre le contrôle total du drone en utilisant des commandes simples et bien documentées. Les utilisateurs peuvent envoyer des instructions via Wi-Fi en établissant une connexion avec le drone, et en exécutant des commandes pour les actions telles que le décollage, l'atterrissage, le mouvement dans différentes directions, et l'exécution de figures comme des flips.

Pour programmer ces commandes, Python est souvent utilisé en raison de sa simplicité et de la large communauté de développeurs. La bibliothèque DJITelloPy, qui facilite l'interaction avec le drone, est l'une des plus populaires pour ce type de projet. Cette bibliothèque permet de :

- Envoyer des commandes de base comme `takeoff`, `land`, `move_forward`, `rotate`, etc.
- Récupérer des données de télémétrie comme la vitesse, l'altitude, et l'état de la batterie en temps réel.
- Intégrer des modules plus avancés tels que `OpenCV` pour la reconnaissance d'objets ou le traitement d'images en direct depuis la caméra du drone.

Command	Enter SDK mode.
<code>takeoff</code>	Auto takeoff.
<code>land</code>	Auto landing.
<code>streamon</code>	Enable video stream.
<code>streamoff</code>	Disable video stream.
<code>emergency</code>	Stop motors immediately.
<code>up x</code>	Ascend to "x" cm. x = 20-500
<code>down x</code>	down "x" Descend to "x" cm. x = 20-500
<code>left x</code>	Fly left for "x" cm. "x" = 20-500
<code>right x</code>	Fly right for "x" cm. "x" = 20-500
<code>forward x</code>	Fly forward for "x" cm. "x" = 20-500
<code>back x</code>	Fly backward for "x" cm. "x" = 20-500
<code>cw x</code>	Rotate "x" degrees clockwise. "x" = 1-360
<code>ccw x</code>	Rotate "x" degrees counterclockwise. "x" = 1-360
<code>flip x</code>	Flip in "x" direction. "l" = left "r" = right "f" = forward "b" = back

Figure 2 : Ressource SDK

L'utilisation du SDK Tello est une première étape essentielle pour prendre en main le contrôle des drones, car elle permet une compréhension fine des commandes de vol et de leur impact sur les performances du drone.

Une fois la connexion établie et les premières commandes envoyées, il devient possible de développer des algorithmes plus complexes pour automatiser les missions de vol.

Conception et impression 3D des cadres de protection

1. Outils et logiciels utilisés pour la conception

Sur l'application Blender, on a créé une conception et une impression 3D de cadres de protection pour le drone Tello. Les cadres sont destinés à protéger les composants du drone, notamment les hélices, en cas de collision ou d'utilisation dans des environnements où des chocs peuvent survenir. Ces structures renforcent la sécurité du drone tout en minimisant les dommages potentiels pendant son fonctionnement.

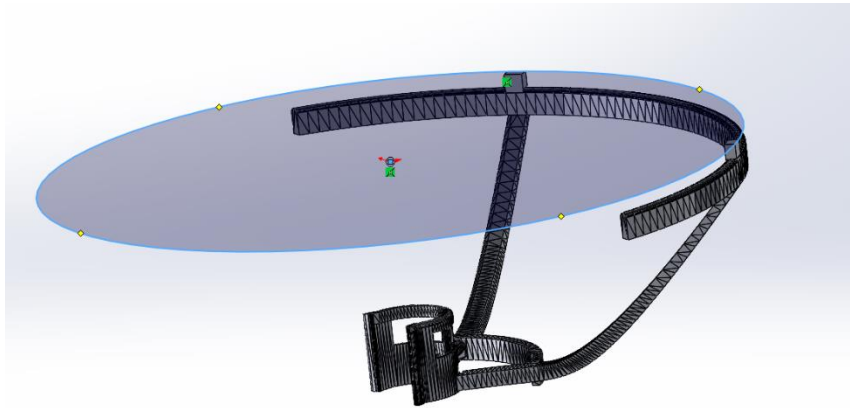


Figure 3 : Vue de profil de la garde

Voici une vue 3D d'une conception semi-transparente d'une structure ovale avec un cadre en forme d'arc autour. Cette structure est faite pour protéger les hélices ou les composants d'un drone contre les chocs.

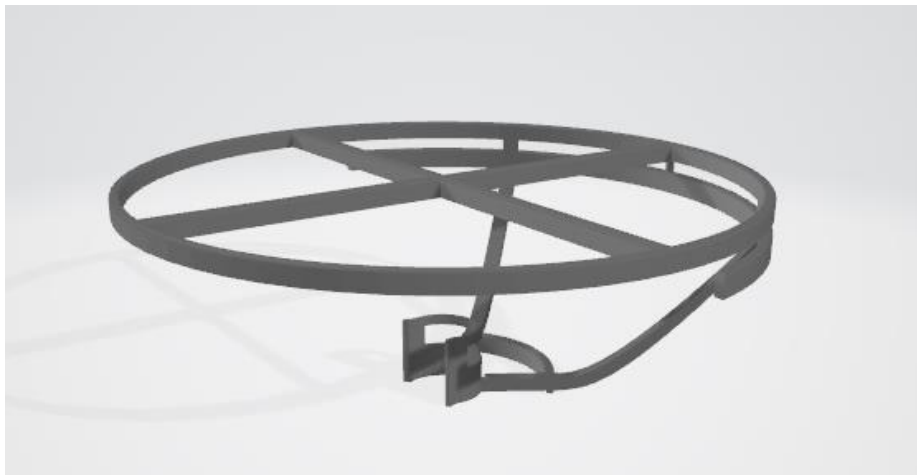


Figure 4 : Conception réelle de la garde

Il s'agit d'un schéma technique montrant les dimensions du cadre de protection. Le cadre est circulaire avec un diamètre mesuré à environ 200 mm, avec des renforts

disposés en croix à l'intérieur pour plus de rigidité. Les cotes indiquées permettent de visualiser les proportions exactes du cadre et des éléments de support.

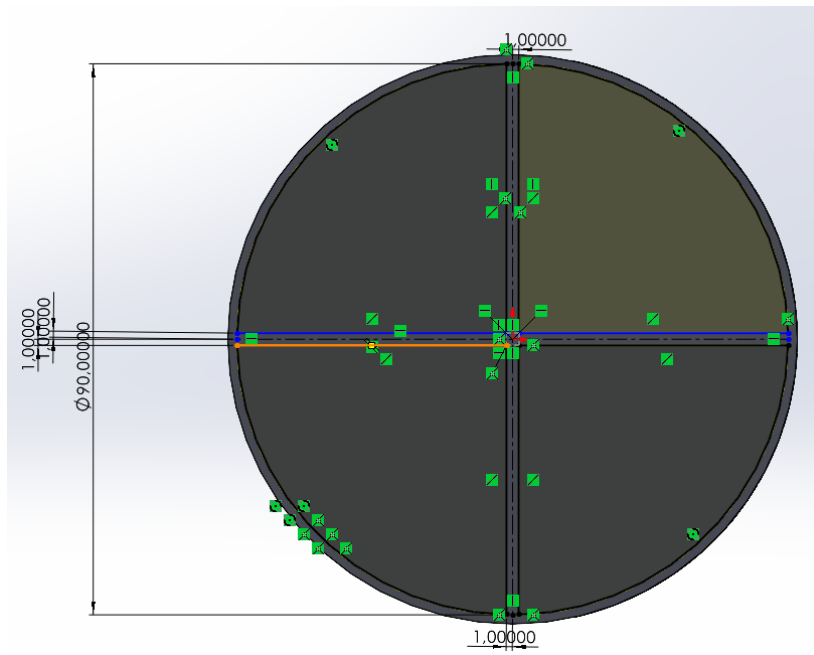


Figure 5 : Dimension de la garde

Cette image montre un prototype 3D du cadre de protection. On voit clairement le cadre circulaire renforcé avec des supports courbés en dessous. Ces supports sont conçus pour attacher le cadre au drone, assurant qu'il reste bien en place tout en protégeant les composants critiques comme les hélices.

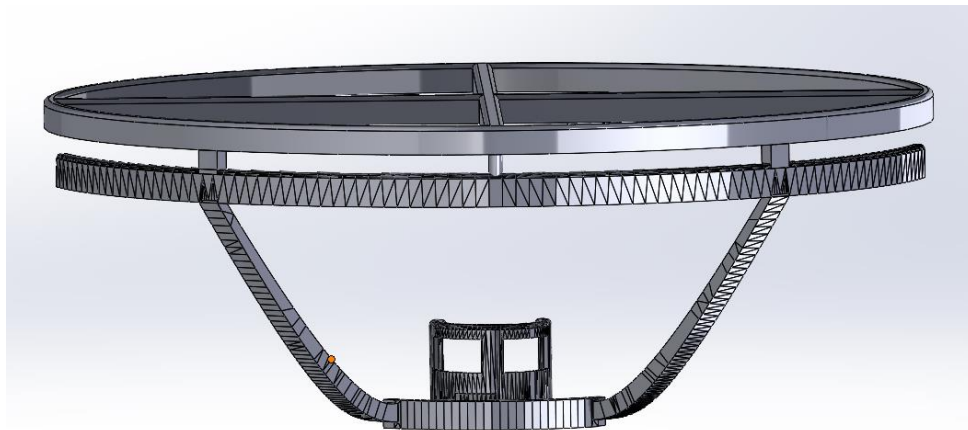


Figure 6 : Prototype de la garde

2. Processus d'impression 3D : choix des matériaux et validation des modèles

Pour l'impression 3D du cadre de protection, nous avons utilisé la technologie FDM (Fused Deposition Modeling), qui permet de produire des pièces solides et précises à faible coût. Avant l'impression, nous avons conçu le modèle sur Blender et SolidWorks, puis effectué des simulations pour valider sa résistance et son ajustement

sur le drone. Une fois le design finalisé, nous avons exporté le fichier au format STL et préparé l'impression avec un slicer tel que Cura ou PrusaSlicer.

Choix des matériaux

Nous avons opté pour le *PLA (Acide Polylactique)*, un polymère couramment utilisé en impression 3D pour ses nombreux avantages.

Choix des matériaux	Type de matériau - Pourquoi ce choix ?	Pourquoi avoir choisi le PLA ?
PLA (Acide Polylactique)	Facile à imprimer, biodégradable, rigide et économique. Offre un bon compromis entre solidité et flexibilité.	Le PLA est un matériau idéal pour ce type de projet. Il est facile à imprimer, ne nécessite pas de températures extrêmes et offre une bonne rigidité, essentielle pour un cadre de protection.

Tableau 2 : Matériau utilisé pour le drone

D'un point de vue environnemental, le PLA est issu de ressources renouvelables (amidon de maïs, canne à sucre) et présente une empreinte carbone plus faible que d'autres plastiques comme l'ABS. Il est biodégradable dans des conditions industrielles, ce qui en fait un choix plus responsable.

Sur le plan financier, le PLA est un matériau abordable, avec un coût d'environ 20 à 30 €/kg, permettant d'imprimer plusieurs prototypes sans coût excessif.

Imprimante utilisée et paramètres d'impression

L'impression a été réalisée avec une Creality CR-10 Smart Pro, une imprimante 3D grande taille et haute précision, équipée d'un cadre métallique robuste et d'un plateau chauffant en verre pour une meilleure adhérence. Elle permet d'imprimer des pièces de grandes dimensions avec une bonne finition.

Les paramètres d'impression étaient :

- Hauteur de couche : 0.2 mm (compromis entre qualité et vitesse)
- Remplissage : 20 % (suffisant pour une bonne rigidité sans trop alourdir la pièce)
- Température d'extrusion : 200°C
- Température du plateau : 60°C
- Vitesse d'impression : 50 mm/s

Après impression, nous avons procédé à plusieurs tests :

1. Test d'assemblage : Vérification que le cadre s'adapte parfaitement au drone Tello.



Figure 7 : Nouvelle protection en PLA

2. Test de résistance : Simulation de chocs pour observer la durabilité du cadre.

Les résultats ont confirmé que le cadre en PLA imprimé avec la Creality CR-10 Smart Pro remplissait bien ses fonctions : protection efficace des hélices, légèreté et solidité. En cas de besoin, nous pourrions ajuster certains paramètres ou tester d'autres matériaux pour améliorer la performance du cadre.

Développement de systèmes de navigation autonome

4.1. Suivi de trajectoire

Pour piloter le drone, nous utilisons une interface graphique que l'on a développé en python. Le but est de positionner des points pour créer une trajectoire de façon automatique. Lorsque les points sont placés, le drone décolle et suit la trajectoire en se déplaçant d'un point à l'autre en ajustant son orientation.

```
if len(points) >= 2 and current_point_index < len(points) and isFlying:
```

Vérification des conditions

- Le drone ne doit bouger que s'il y a au moins **deux points** à suivre.
- `current_point_index < len(points)`: Vérifie que le drone n'a pas fini sa trajectoire.
- `isFlying`: Vérifie que le drone a bien décollé.

```
x_current, y_current = points[current_point_index - 1]
x_target, y_target = points[current_point_index]
```

Récupération des coordonnées

- `x_current, y_current` : Position actuelle du drone (point précédent).
- `x_target, y_target` : Prochain point à atteindre.

```
new_angle = calculate_rotation_angle(x_current, y_current, x_target, y_target)
```

Calcul de l'angle de rotation

- `calculate_rotation_angle()` détermine **combien** le drone doit tourner pour être orienté vers le prochain point.

```
distance = calculate_distance(x_current, y_current, x_target, y_target)
```

Calcul de la distance à parcourir

- `calculate_distance()` calcule la distance entre les deux points.

```
if new_angle < 0:
    tello.rotate_counter_clockwise(abs(new_angle))
else:
    tello.rotate_clockwise(new_angle)

current_angle += new_angle
```

Rotation du drone

- Si `new_angle` est **négatif**, le drone tourne à gauche (*counter-clockwise*).
- Sinon, il tourne à droite (*clockwise*).

1. Mise à jour de l'angle du drone

```
current_angle += new_angle
```

- Met à jour `current_angle` pour conserver l'orientation correcte du drone.

Avancer jusqu'au prochain point

```
tello.move_forward(int(distance))
```

- Une fois le drone bien orienté, il avance de la distance calculée.

Passage au point suivant

```
current_point_index += 1
```

- Incrémente l'index pour traiter le point suivant.

Pourquoi ce choix de logique ?

Utilisation de `calculate_rotation_angle()` et `calculate_distance()`

Plutôt que d'intégrer directement les formules dans `move_drone()`, on garde une structure modulaire. Cela rend le code plus lisible et réutilisable.

- Vérification `len(points) >= 2`

Cela empêche le drone d'essayer de bouger sans avoir de destination.

- Mise à jour de `current_angle`

Il est crucial de suivre l'orientation actuelle du drone, car chaque rotation doit être calculée par rapport à son angle précédent.

- Utilisation de `current_point_index`

Permet au drone de suivre chaque point de la trajectoire dans l'ordre, sans revenir en arrière.

- Utilisation d'une boucle infinie `while True`

Cette approche assure que le drone continue à suivre tous les points jusqu'à la fin de la liste.

Ce code met en place une navigation en points-clés en s'assurant que le drone tourne dans la bonne direction, avance jusqu'au point suivant et répète l'opération jusqu'à atteindre la fin de la trajectoire

4.2. Reconnaissance d'objets : principes et algorithmes utilisés

Le script permet de **détecter et suivre un visage** en utilisant un **drone Tello** et **OpenCV**. Voici les points essentiels :

1. Connexion au drone

- Le drone est connecté (Tello()), et son niveau de batterie est affiché.
- La caméra du drone est activée avec `tello1.streamon()`.

2. Détection du visage avec OpenCV

- Le modèle **MediaPipe** est utilisé pour détecter les visages.
- La détection est effectuée en convertissant l'image en niveaux de gris et en utilisant MediaPipe
- Le visage détecté est entouré d'un rectangle vert, et un point vert est placé au centre.

3. Suivi du visage et contrôle du drone

- La fonction `trackFace()` ajuste les mouvements du drone en fonction de la position du visage.

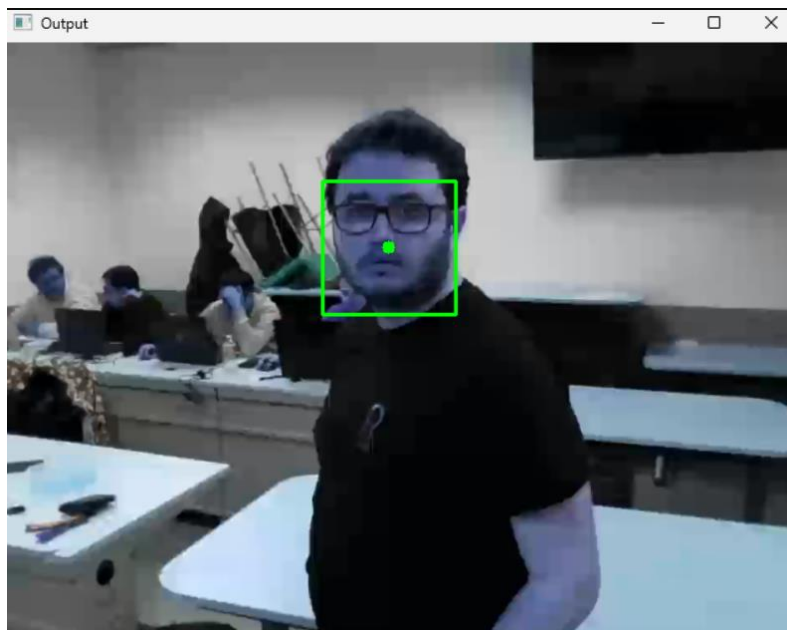


Figure 8 : Image envoyée de la caméra du drone

Le suivi du visage est assuré par une fonction qui ajuste les mouvements du drone en fonction de la position et de la taille du visage détecté.

Si le visage est bien centré et dans une plage de distance adéquate, le drone reste immobile.

Si le visage est trop loin, le drone avance, et s'il est trop proche, il recule.

De même, si le visage est décalé sur la gauche ou la droite, le drone effectue une rotation pour recentrer l'image.

4. Acquisition et affichage des images

- La caméra du drone fournit une image en continu, qui est redimensionnée (`cv2.resize()`).

- L'image est traitée et affichée avec `cv2.imshow()`.

5. Arrêt du programme

L'ensemble du processus fonctionne en boucle, avec l'image traitée affichée en direct sur l'écran. L'exécution du programme continue jusqu'à ce que l'utilisateur appuie sur la touche 'q', ce qui entraîne l'arrêt du flux vidéo et la fermeture des fenêtres d'affichage.

4.4. Intégration des systèmes

L'intégration des systèmes au drone concerne plusieurs aspects :

1. *Connexion et initialisation du drone (`tello.connect()`)*
2. *Lancement des différents threads (déplacement, arrêt d'urgence, décollage)*
3. *Vérification du niveau de batterie*
4. *Simulation et affichage de la trajectoire avec Pygame*

`Tello()` est une instance d'un objet qui permet de contrôler le drone via la bibliothèque `djitellopy` et `tello.connect()` établit la connexion entre le PC et le drone via Wi-Fi.

```
tello = Tello()  
tello.connect()
```

Initialisation de l'interface graphique

```
pygame.init()  
  
WIDTH, HEIGHT = 800, 600  
win = pygame.display.set_mode((WIDTH, HEIGHT))  
pygame.display.set_caption("Contrôle du Drone avec Trajectoire Numérotée")
```

Pygame est utilisé pour créer une interface où l'utilisateur peut définir la trajectoire du drone. Voici les commandes utilisées :

- `pygame.display.set_mode((WIDTH, HEIGHT))` définit la taille de la fenêtre.
- `pygame.display.set_caption()` permet d'afficher un titre.

Lancement des threads

L'utilisation des threads est utilisée pour gérer plusieurs tâches en parallèle. Voici les threads utilisés :

```
emergency_thread = Thread(target=emergency_stop)
emergency_thread.daemon = True
emergency_thread.start()

drone_movement_thread = Thread(target=move_drone)
drone_movement_thread.daemon = True
drone_movement_thread.start()

drone_takeoff_thread = Thread(target=takeoff_drone)
drone_takeoff_thread.daemon = True
```

- emergency_thread écoute la touche Échap pour un atterrissage d'urgence.
- drone_movement_thread gère le déplacement du drone.
- drone_takeoff_thread démarre le décollage.

Boucle principale de simulation

```
while True:
    pygame.time.delay(50)

    for event in pygame.event.get():
        if event.type == pygame.MOUSEBUTTONDOWN:
            x, y = pygame.mouse.get_pos()
            points.append((x, y))
            if len(points) == 1:
                drone_x, drone_y = x, y
                drone_takeoff_thread.start()
                current_point_index += 1
            print(points)
            print(points[current_point_index-1])
```

Le principal rôle de l'utilisateur est de cliquer sur l'interface pour définir les points de la trajectoire. Le premier point est le point de départ du drone, qui déclenche le décollage (drone_takeoff_thread.start()).

Affichage et mise à jour graphique

```
win.fill(WHITE)
pygame.draw.rect(win, BLACK, (50, 50, WIDTH - 100, HEIGHT - 100), 5)

for i, point in enumerate(points):
    pygame.draw.circle(win, BLACK, point, 5)

draw_point_numbers()

if len(points) >= 2:
    pygame.draw.lines(win, RED, False, points, 2)

win.blit(drone_image, (drone_x, drone_y))
pygame.display.update()
```

L'affichage des éléments graphiques se fait en plusieurs étapes. Tout d'abord, la commande `win.fill(WHITE)` est utilisée pour effacer l'écran et préparer un nouvel affichage. Ensuite, la fonction `pygame.draw.rect()` permet d'afficher une zone de vol délimitée.

Pour représenter chaque point, la fonction `pygame.draw.circle()` est utilisée, tandis que `draw_point_numbers()` sert à numéroté ces points afin d'améliorer la clarté de l'affichage. Afin de visualiser la trajectoire, les points sont reliés entre eux grâce à la fonction `pygame.draw.lines()`.



L'image du drone est affichée à l'écran à l'aide de `win.blit(drone_image, (drone_x, drone_y))`, qui positionne l'image du drone aux coordonnées spécifiées.



De plus, une LED a été ajoutée sur l'interface graphique pour indiquer si la batterie est utilisable ou non et si le drone est connecté ou non.

Pour pousser le projet un peu plus loin, on a créé le vol en essaim. Cette approche permet à un groupe de drones de se déplacer de manière coordonnée en suivant une trajectoire définie par une personne via l'interface graphique.

Principe de fonctionnement

L'essaim de drones est contrôlé par un système central qui analyse la trajectoire définie par l'utilisateur sur une interface graphique intuitive. Chaque drone reçoit des instructions précises concernant son déplacement et son orientation. Le système utilise des algorithmes de calcul d'angle de rotation, comme celui illustré dans le code fourni, pour ajuster dynamiquement la direction de chaque drone.

```
def calculate_rotation_angle(x_current, y_current, x_next, y_next, current_angle):  
    delta_x = x_next - x_current  
    delta_y = y_next - y_current  
    target_angle = math.atan2(delta_y, delta_x)  
    target_angle_degrees = math.degrees(target_angle)  
    rotation_angle = target_angle_degrees - current_angle  
    rotation_angle = (rotation_angle + 180) % 360 - 90  
    return int(rotation_angle)
```

Interface graphique finale :

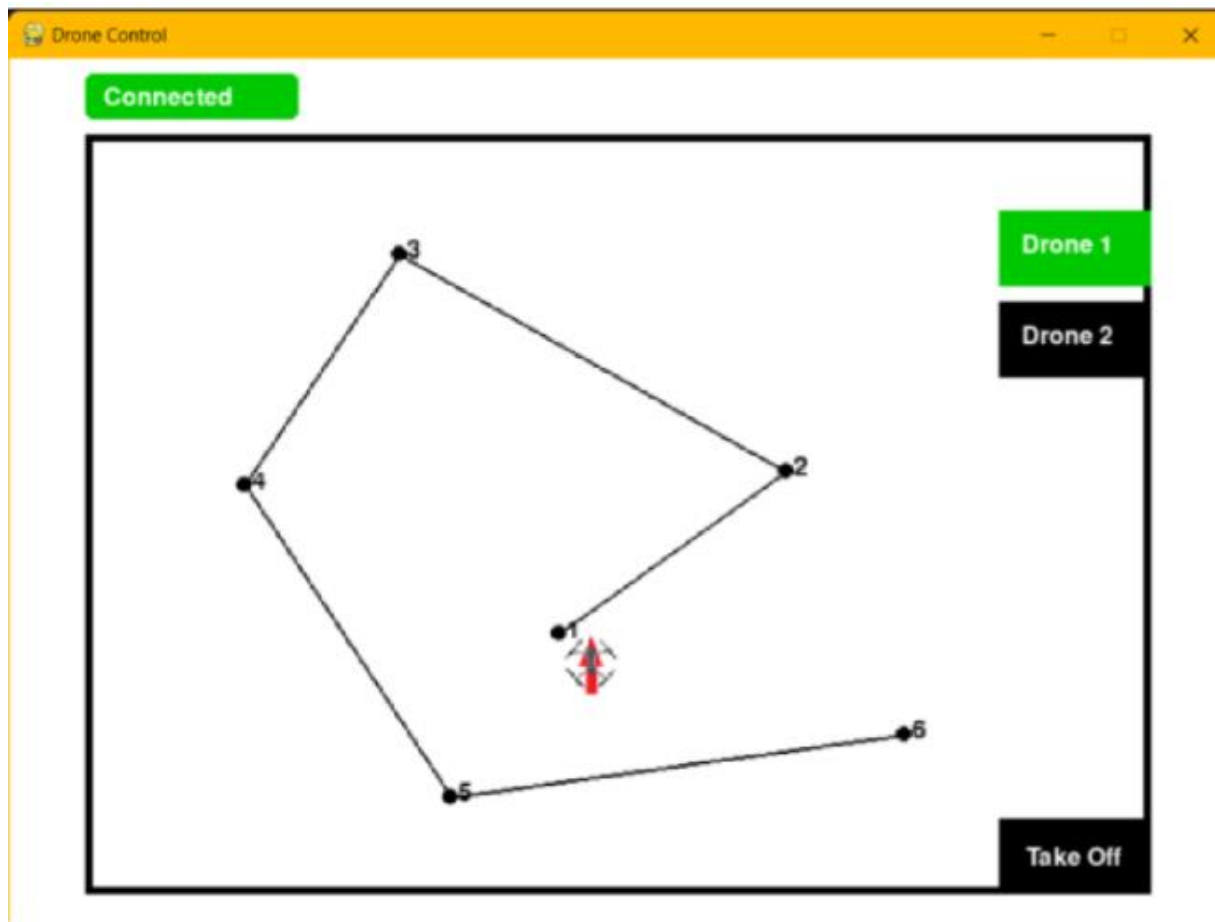


Figure 9 : Interface graphique python

Cette partie intègre le drone avec une interface et une simulation.

Elle assure que le drone se connecte et vérifie sa batterie, que plusieurs tâches s'exécutent en parallèle (mouvement, arrêt d'urgence...), que l'utilisateur définit la trajectoire avec Pygame et enfin que l'interface affiche les points et le chemin suivi sur plusieurs drones.

Implémentation et tests en conditions réelles

5.1. Mise en place des scénarios de vol d'essai

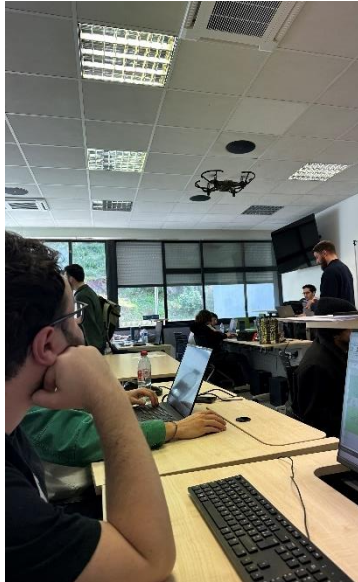


Figure 10 : Mis en situation

Les tests en conditions réelles ont été réalisés en salle de classe, un environnement contrôlé permettant de valider le bon fonctionnement du système avant une utilisation en extérieur.

Plusieurs scénarios ont été définis pour tester les capacités de navigation et de suivi de trajectoire du drone.

Petit à petit, de nouveaux scénarios vont être créés pour optimiser le projet et définir des objectifs.

Scénario	Type	Objectif
<u>Scénario 1</u>	Test basique de déplacement (gauche, droite)	S'assurer que le drone suit correctement une trajectoire simple. - Le drone décolle d'un point fixe. - L'utilisateur place 3 points en ligne droite. - Le drone suit les points et atterrit automatiquement.
<u>Scénario 2</u>	Test de navigation avec rotation	Évaluer la capacité du drone à ajuster son orientation. Les points sont placés en forme de L. Le drone doit tourner correctement pour suivre la trajectoire
<u>Scénario 3</u>	Test de trajectoire complexe	Vérifier la gestion d'une trajectoire en diagonale. Plusieurs points sont placés en alternance.

		Le drone doit ajuster ses rotations et suivre le chemin précisé.
<u>Scénario 4</u>	Test d'urgence	Évaluer la réactivité du système d'arrêt d'urgence. - Le drone décolle et commence son trajet. - L'utilisateur appuie sur la touche echap ou le bouton arrêt pour arrêter le drone. - Le drone doit immédiatement atterrir en sécurité.
<u>Scénario 5</u>	Test de la caméra	Utilisation de la caméra du drone pour diriger le drone.
<u>Scénario 6</u>	Faire voler plusieurs drones en même temps	- On utilise deux drones décollent en même temps - Effectue le même parcours

5.2. Analyse des performances et résultats obtenus

L'analyse des tests a permis d'évaluer la précision de navigation, la stabilité du vol et la rapidité d'exécution des commandes.

- **Précision du suivi de trajectoire :**
 - Dans les scénarios simples (1 et 2), le drone a suivi les points avec une marge d'erreur inférieure à 5 cm.
 - Dans le scénario diagonale (3), de légers ajustements ont été observés, mais le comportement restait conforme aux attentes.
 - Dans le scénario (4), on ajoute un bouton d'arrêt pour éviter l'utilisation du clavier.
 - Dans le scénario (5), on visualise l'environnement dans lequel le drone se trouve.
 - Dans le scénario (6), on crée un parcours où 2 drones vont suivre la trajectoire en même temps.



Figure 11 : Pilotage de 2 drones

- **Réactivité aux changements d'orientation :**
 - Le calcul d'angle s'est montré efficace, avec une rotation fluide et adaptée.
- **Gestion des situations d'urgence :**
 - Le drone a immédiatement réagi à la pression de la touche Échap, prouvant l'efficacité du thread d'urgence.

Les tests ont confirmé la fiabilité du système, bien que certaines améliorations soient possibles.

5.3. Réglages et optimisations pour les améliorations futures

Plusieurs points d'amélioration ont été identifiés pour donner suite aux tests :

1. **Optimisation de la précision des rotations :**
 - Actuellement, les rotations peuvent entraîner un léger décalage. Une amélioration des calculs d'angles pourrait renforcer la stabilité du vol.
2. **Correction des légers décalages dans les trajectoires complexes :**
 - Ajouter une réévaluation de la position après chaque déplacement pour limiter les erreurs cumulatives.
3. **Gestion dynamique de la vitesse :**
 - Adapter la vitesse du drone en fonction de la distance entre deux points pour une meilleure fluidité de mouvement.
4. **Amélioration de l'interface graphique :**
 - Ajouter une visualisation en temps réel de la progression du drone sur la trajectoire.

Avec ces améliorations, le système pourrait être encore plus robuste et précis, facilitant l'expérimentation en environnements plus variés.

Conclusion

6.1. Résumé des résultats

Ce projet de développement d'un système de navigation autonome pour les drones Tello a permis d'explorer différents aspects techniques, allant de la prise en main du drone, de son SDK, à la conception de cadres de protection en impression 3D, en passant par le développement et l'implémentation d'algorithmes de navigation autonome grâce à la reconnaissance faciale. L'utilisation du SDK Tello a permis de programmer le drone en Python, offrant un contrôle précis des mouvements et facilitant l'intégration de modules de reconnaissance d'objets basés sur OpenCV. Par ailleurs, la mise en place d'un suivi de trajectoire a été réalisée avec succès, permettant au drone de se déplacer de manière autonome en suivant un ensemble de points prédéfinis.

Séance de projet :	07/10/2024 (Journée)	14/10/2024 (Journée)	25/02/2025	04/03/2025	11/03/2025	18/03/2025 (Journée)	24/03/2025	25/03/2025 (Journée)
Présentation projet								
Conception et impression 3D								
Mise en place de scénario								
Suivi de trajectoire								
Reconnaissance d'objet et de visage								
Vol en essaim								
Réglage et optimisation (Code)								
Perspectives des résultats								
Rédaction du rapport								
Création Power Point								

Figure 12 : Planification GANTT

Enfin, les tests en conditions réelles ont confirmé la fiabilité du système, bien que certaines optimisations soient encore nécessaires pour améliorer la stabilité du vol et la réactivité du drone face aux obstacles.

6.2. Perspectives d'évolution du projet

Plusieurs axes d'amélioration peuvent être envisagés pour la suite du projet. Tout d'abord, l'intégration de capteurs supplémentaires, tels que des capteurs ultrasoniques ou LIDAR, permettrait d'améliorer la détection des obstacles et d'affiner la navigation en environnements complexes.

Ensuite, l'optimisation des algorithmes de reconnaissance d'objets pourrait renforcer la capacité du drone à identifier et suivre des cibles en mouvement avec une plus grande précision.

Par ailleurs, l'utilisation d'une intelligence artificielle plus avancée, basée sur des réseaux de neurones, permettrait d'améliorer la prise de décision en vol et d'adapter dynamiquement les trajectoires en fonction des conditions de l'environnement. Enfin,

une perspective intéressante serait le déploiement du projet sur plusieurs drones afin d'explorer les possibilités de vol en essaim, où plusieurs drones interagissent et collaborent pour réaliser des missions plus complexes.

En somme, ce projet constitue une base solide pour le développement de systèmes de navigation autonome appliqués aux drones, avec un potentiel d'évolution significatif vers des applications avancées telles que la surveillance, la cartographie et l'exploration autonome.

Ressources

Dimensions :

https://www.researchgate.net/figure/Size-of-the-DJI-RYZE-TELLO-drone_fig8_358811122

SDK :

<https://dl-cdn.ryzrobotics.com/downloads/Tello/Tello%20SDK%202.0%20User%20Guide.pdf>

Projet :

https://www.researchgate.net/publication/367093080_Real-Time_Human_Motion_Tracking_by_Tello_EDU_Drone
<https://www.youtube.com/watch?v=c21RSkkwA04>

<https://theses.hal.science/tel-00954584/>

<https://blog.studiosport.fr/ryze-tello-dji-test-presentation/>

<https://github.com/murtazahassan/Tello-Object-Tracking>

<https://github.com/Matthewjsiv/Person-Tracking-Tello-Drone>

<https://jckantor.github.io/CBE30338/B.02-Visual-Tracking-of-an-Object-with-a-Drone.html>